
CPU SCHEDULER SIMULATION

TECHNICAL REPORT

• **Seyed Mohammad Mahdi Razavi**
Department of Computer Science
Shahid Beheshti University
Tehran, Iran

ABSTRACT

This project presents the design and implementation of a comprehensive CPU scheduling simulator capable of modeling and evaluating multiple classical scheduling algorithms. The simulator supports First-Come First-Serve (FCFS), Shortest Job First (SJF), Round Robin (RR), Priority Scheduling with aging, and a fully functional Multilevel Feedback Queue (MLFQ).

The system is designed in a modular and scheduler-agnostic manner, allowing each scheduling policy to express its behavior while the simulator manages time progression, context switching, and event handling.

To achieve realistic behavior, advanced features such as time slicing, aging to prevent starvation, I/O blocking, promotion after I/O completion, arrival-based preemption, and periodic priority boosting were incorporated where applicable.

The simulator was evaluated under CPU-bound, I/O-bound, and mixed workloads, and key performance metrics including turnaround time, waiting time, response time, CPU utilization, throughput, and fairness were collected.

The experimental results align closely with theoretical expectations, demonstrating the trade-offs between efficiency, responsiveness, and fairness among different scheduling strategies. This project provides a practical and extensible framework for analyzing CPU scheduling policies and understanding their real-world implications.

1 Introduction

CPU scheduling is one of the most fundamental responsibilities of an operating system, directly influencing system performance, responsiveness, and fairness among processes. Since different workloads exhibit different execution characteristics, no single scheduling algorithm is optimal for all scenarios. As a result, understanding the behavior and trade-offs of various scheduling strategies is a core topic in operating systems education and research.

The objective of this project is to design and implement a flexible CPU scheduling simulator that allows for systematic comparison of multiple scheduling algorithms under diverse workload conditions. The simulator supports both non-preemptive and preemptive policies, ranging from simple algorithms such as FCFS to more advanced adaptive strategies like MLFQ.

A key design goal of the simulator is separation of concerns: scheduling policies are encapsulated within independent scheduler classes, while the simulator itself is responsible for managing the global clock, handling events such as process arrivals and I/O completions, and recording execution traces. This design enables extensibility and ensures that new scheduling algorithms can be added with minimal changes to the core simulation engine.

In addition to classical implementations, the project enhances Priority Scheduling with aging to prevent starvation and extends MLFQ with realistic features such as multiple priority levels, variable time quanta, promotion after I/O, arrival-based preemption, and periodic priority boosting. The simulator is evaluated using synthetic workloads representing CPU-bound, I/O-bound, and mixed process behaviors, allowing a detailed analysis of scheduler performance across different system conditions.

2 PCB(Process Control Block)

PCB is an abbreviation for Process Control Block. It is a data structure in the operating system that stores information about a process. In this design, the PCB includes:

Pid : which is the process identifier

Arrival : which represents the time the process enters the system

Cpu_burst : which indicates the amount of time the process needs the CPU

Io_burst : which shows the duration of I/O operations

Priority : which indicates the priority of the process

State : which represents the current status of the process

The state of a process can be one of the following: New, Ready, Running, Waiting, or Terminated.

The statistical variables start_time, completion_time, waiting_time, and response_time are also used to evaluate the performance of the process.

3 Event

We use events to simulate process scheduling in an operating system. A comparison method is implemented whose purpose is to determine, when comparing two events, which one occurs earlier. This is useful in queue-based data structures. In this case, time is used as the comparison criterion, and the process with the smaller time has higher priority.

4 FCFS(First Come First Serve)

The FCFS scheduling algorithm stands for First Come, First Served, meaning that the process which enters the queue earlier gets the CPU earlier.

A deque is a double-ended and very fast queue that is commonly used in queue-based algorithms. The reason for using a deque is that it works from both ends, allowing the first entered process to be removed easily, since processes receive the CPU in the order they arrive. Other queue-based structures do not provide this capability as efficiently.

The time parameter represents the current time of the algorithm. Although it is not directly used in FCFS, it is maintained for consistency with other scheduling algorithms. If the queue is not empty, the first process is removed and returned.

In FCFS, each process runs without preemption and holds the CPU until it finishes its execution. Therefore, the time slice in FCFS is equal to the remaining execution time of the process.

5 SJF(Shortest Job First)

The SJF (Shortest Job First) scheduling algorithm works in such a way that the process with the shorter execution time receives the CPU earlier.

SJF requires a priority queue so that the process with the highest priority is always placed at the front of the queue. In this implementation, tuples are used so that processes can be compared based on the elements of the tuple.

The main criterion in SJF is ordering processes from the shortest execution time to the longest. Additionally, if two processes have equal execution times, they are ordered based on their arrival time. This ensures that the process with the smallest remaining time is always selected first, and in case of a tie, the order of arrival is preserved.

The time parameter represents the current time of the simulation for this algorithm. Here, the smallest process in the queue is removed, and its PCB is returned.

The SJF algorithm is non-preemptive, meaning that once a process starts execution, it keeps the CPU until it finishes.

6 Priority Scheduling

The goal of the Priority scheduling algorithm is to schedule processes based on their priority.

In this algorithm, the process with a higher priority (a smaller numerical value) is executed earlier.

An aging method is implemented to prevent starvation. With this approach, processes that have been waiting for a longer time gradually receive a higher priority.

The priority scheduler maintains stable and deterministic ordering even when multiple processes share the same priority, using arrival time and process ID as tie-breakers. Aging adjusts priorities without disrupting queue consistency.

7 Round Robin

The Round Robin (RR) scheduling algorithm is one of the most important algorithms in time-sharing systems. It belongs to the preemptive category.

This algorithm, similar to FCFS, uses a FIFO queue (First In, First Out), but with one important difference: each process is allowed to run only for a fixed amount of time called a quantum. After the quantum expires, the process is preempted and placed back into the queue.

As a result, processes are executed in the order in which they enter the queue. The execution behavior is as follows: if a process needs less time than the quantum, it runs only for its required time and then terminates without returning to the queue. If the remaining execution time of a process is greater than the quantum, it runs for exactly one quantum and is then added back to the queue to continue execution later.

8 Multilevel Feedback Queue (MLFQ)

The Multilevel Feedback Queue (MLFQ) scheduler implemented in this project represents an advanced and adaptive CPU scheduling strategy designed to balance responsiveness, throughput, and fairness. Unlike static scheduling algorithms, MLFQ dynamically adjusts process priorities based on their runtime behavior, making it particularly effective for systems with mixed workloads.

In this implementation, multiple priority levels are maintained, each associated with a distinct time quantum. Newly arriving processes are placed in the highest-priority queue to ensure fast initial response. CPU-bound processes that exhaust their time quantum are gradually demoted to lower-priority queues with longer time slices, while I/O-bound processes are promoted when they return from I/O, favoring interactive behavior.

To prevent starvation, the scheduler periodically performs a priority boost, moving all waiting processes back to the highest priority level. This mechanism ensures long-running processes eventually receive CPU time, even in the presence of frequent short or interactive tasks. The MLFQ scheduler interacts closely with the simulator to handle quantum expiration, I/O completion, preemption by higher-priority queues, and global time-based boosting, resulting in a realistic and comprehensive implementation of this complex scheduling policy.

9 Process Generator

The Process Generator module is designed to produce synthetic workloads that simulate diverse operating system environments. Each generated process is assigned an arrival time, CPU burst length, I/O characteristics, and static priority, encapsulated within a Process Control Block (PCB).

Process arrival times are generated using an exponential distribution, modeling the stochastic nature of real-world process arrivals. Three workload profiles are supported:

CPU-bound Workload (workload = "cpu")

In CPU-bound workloads, processes require long CPU bursts with minimal I/O activity. This configuration stresses the scheduler's ability to efficiently handle long-running tasks and highlights differences in waiting time and turnaround performance.

CPU burst: large (e.g., 80–120 time units)

I/O burst: small (e.g., 10–20 time units)

Behavior: long uninterrupted CPU execution

I/O-bound Workload (workload = "io")

I/O-bound workloads model interactive or I/O-intensive applications that frequently block for I/O operations. These workloads emphasize responsiveness and favor schedulers that quickly return processes to the CPU after I/O completion.

CPU burst: small (e.g., 10–20 time units)

I/O burst: large (e.g., 80–120 time units)

Behavior: frequent blocking and short CPU usage

Mixed Workload (workload = "mixed")

The mixed workload combines characteristics of both CPU-bound and I/O-bound processes, providing a realistic and heterogeneous environment. CPU burst times are generated using a Gaussian distribution to introduce variability.

CPU burst: medium, normally distributed

I/O burst: moderate and variable

Behavior: diverse execution patterns

By supporting multiple workload types, the process generator enables comprehensive and fair evaluation of scheduling algorithms under controlled yet realistic conditions.

10 Simulator

The Simulator module constitutes the core execution engine of the CPU scheduling framework. Its primary responsibility is to emulate the behavior of a uniprocessor operating system by coordinating process arrivals, CPU allocation, context switching, and process completion over simulated time.

The simulator operates using a global logical clock that advances discretely based on scheduling events rather than real time. Process arrivals are modeled using an event-driven approach: all arrival events are inserted into a priority queue ordered by arrival time. At each simulation step, the simulator checks for pending arrivals and moves newly arrived processes into the scheduler's ready structure.

The simulator is designed to be scheduler-independent. It interacts with scheduling algorithms exclusively through a common interface that includes methods for inserting processes into the ready queue, selecting the next process to execute, determining the time slice, handling quantum expiration, and applying aging or priority boosting mechanisms when supported. This design allows the same simulator to correctly support non-preemptive (FCFS, SJF) and preemptive (Round Robin, Priority, MLFQ) scheduling policies.

Context switching overhead is explicitly modeled. When the CPU switches from one process to another, a fixed context switch delay is added to the simulation clock, and the CPU is considered idle during this interval. This provides a more realistic approximation of actual operating system behavior.

During execution, the simulator updates each process's remaining CPU burst, tracks CPU busy time, and records execution intervals in a trace log. Upon completion of a process, its completion time is stored for later statistical analysis. The simulation terminates when all processes have completed execution. By combining event-driven arrivals, scheduler abstraction, and precise time accounting, the simulator provides a robust and extensible platform for evaluating CPU scheduling algorithms.

11 Main Program

The main program acts as the entry point of the simulation framework and coordinates the overall experimental workflow. It initializes random seeds for reproducibility, generates workloads using the process generator, and instantiates multiple scheduling algorithms.

For each workload type, the main program runs independent simulations for all supported schedulers using identical process sets. After each simulation, performance metrics are computed and displayed, ensuring a fair and controlled comparison among scheduling policies.

The program also saves statistical results and execution traces for further analysis and reporting. By automating experiment execution and result collection, the main module enables efficient benchmarking and systematic evaluation of CPU scheduling algorithms under varied workload condition

12 Results

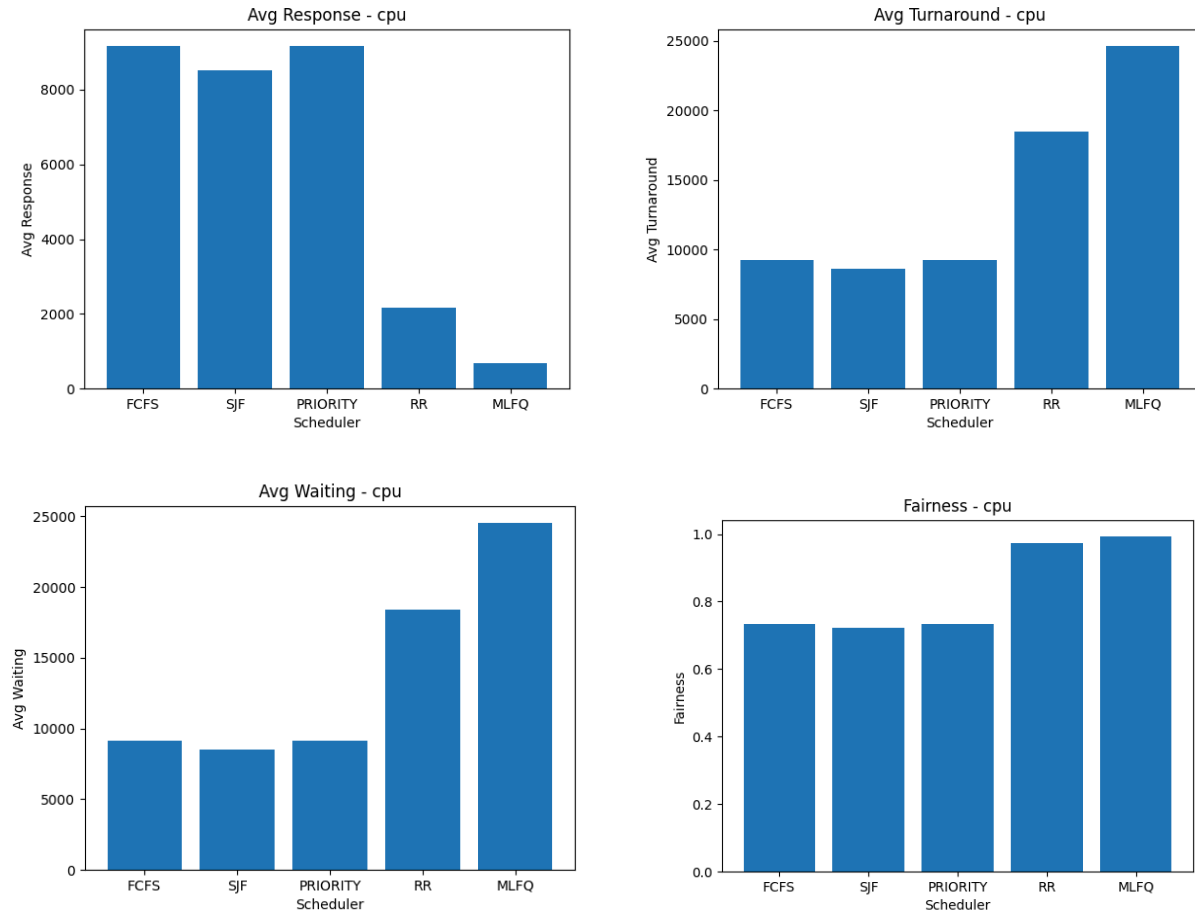


Figure 1 : CPU-Bound Scheduling

	Avg Turnaround	Avg Waiting	Avg Response	CPU Util	Throughput	Fairness
FCFS	9266.73	9166.4	9166.4	98.05032983 141949	0.009772782 799902272	0.733175028 3207104
SJF	8611.875	8511.545	8511.545	98.05032983 141949	0.009772782 799902272	0.723718318 4895476
Priority Scheduling	9266.47	9166.14	9166.14	98.05032983 141949	0.009772782 799902272	0.733041196 9179454
Round Robin	18484.035	8383.705	2178.44	90.18832307 069981	0.008989168 052496742	0.972671761 0444255
MLFQ	24612.89	24512.56	680.075	71.04014727 749062	0.007080648 587410606	0.992562095 8618039

Table 1 : CPU-Bound Scheduling

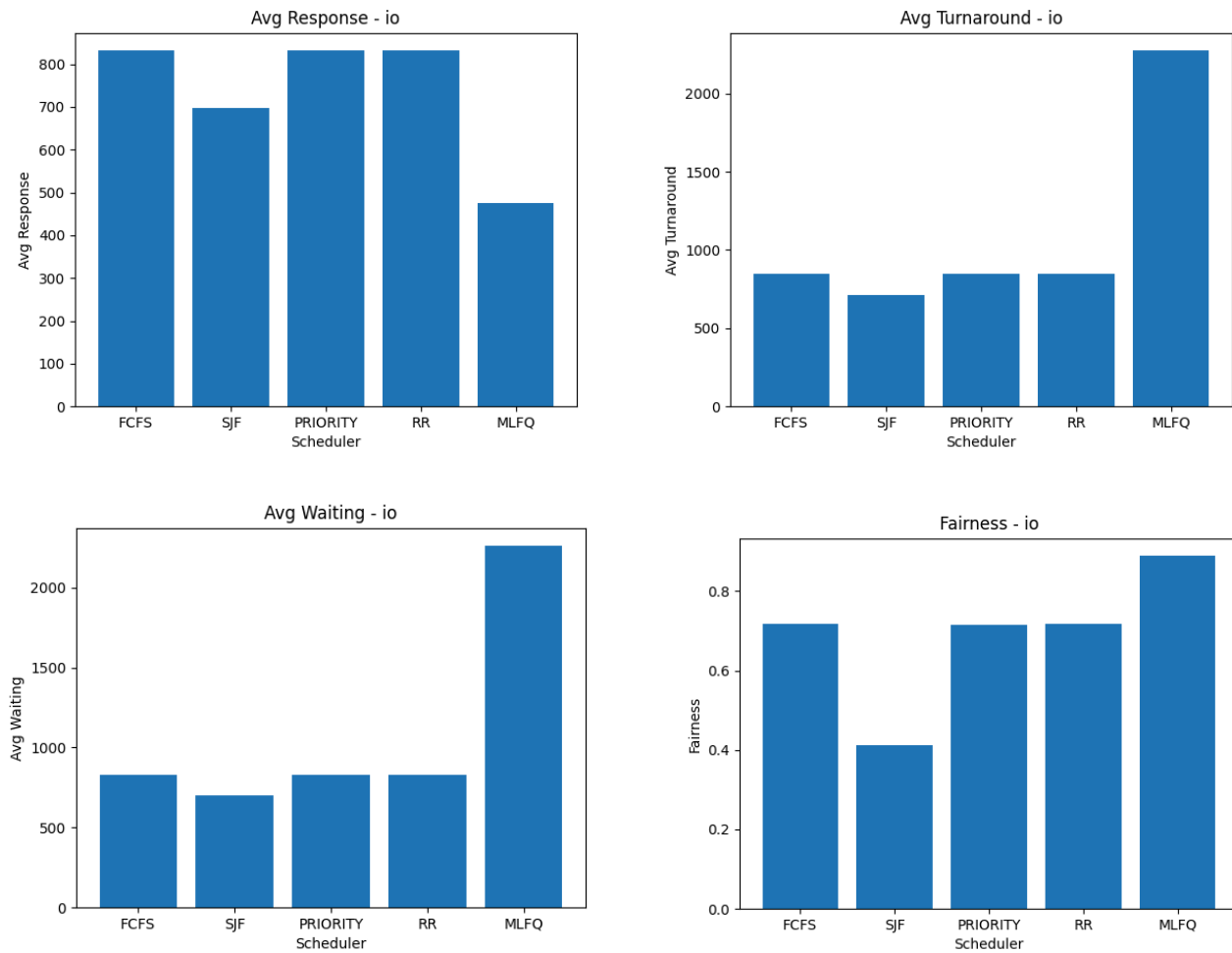


Figure 2 : I/O-Bound Scheduling

	Avg Turnaround	Avg Waiting	Avg Response	CPU Util	Throughput	Fairness
FCFS	846.935	831.49	831.49	88.30760434 53402	0.057175528 87364208	0.718611594 3394812
SJF	713.68	698.235	698.235	88.30760434 53402	0.057175528 87364208	0.411124450 89952776
Priority Scheduling	846.48	831.035	831.035	88.30760434 53402	0.057175528 87364208	0.714929828 0050455
Round Robin	846.935	831.49	831.49	88.30760434 53402	0.057175528 87364208	0.718611594 3394812
MLFQ	2275.115	2259.67	475.985	68.61394935 584185	0.044424700 1332741	0.888636521 2598879

Table 2 : I/O-bound Scheduling

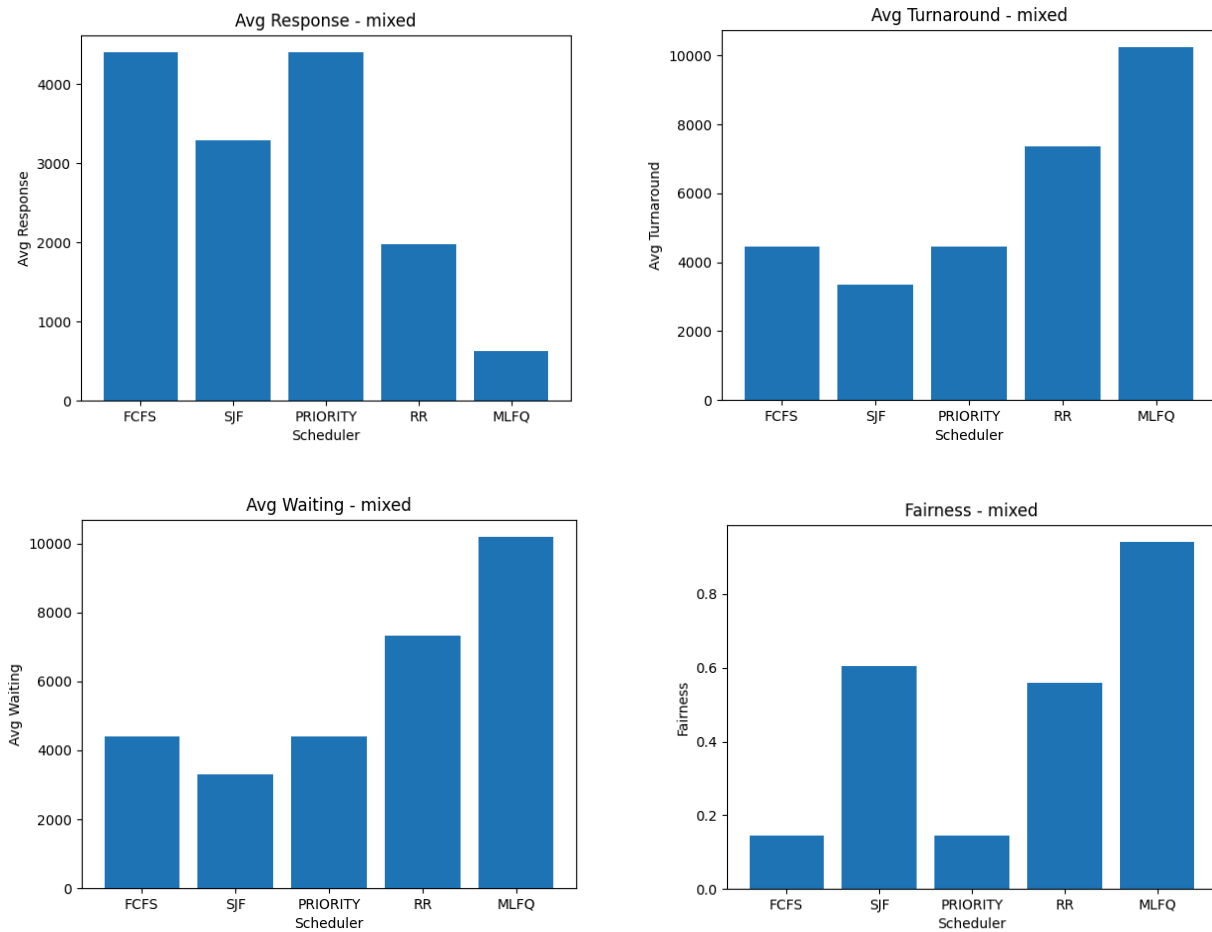


Figure 3 : Mixed-Bound Scheduling

	Avg Turnaround	Avg Waiting	Avg Response	CPU Util	Throughput	Fairness
FCFS	4450.865	4398.985	4398.985	96.14529280 948851	0.018532246 108228317	0.145209761 59107013
SJF	3343.95	3292.07	3292.07	96.14529280 948851	0.018532246 108228317	0.605087577 1700475
Priority Scheduling	4451.67	4399.79	4399.79	96.14529280 948851	0.018532246 108228317	0.145309722 01283934
Round Robin	7364.545	7312.665	1981.78	89.23288613 69109	0.017199862 40110079	0.558256195 1763012
MLFQ	10234.36	10182.48	627.355	68.83375348 28181	0.013267878 466233248	0.940838408 8918411

Table 3 : Mixed-Bound Scheduling

The experimental results align with theoretical expectations. SJF minimizes turnaround time at the cost of fairness, while Round Robin and MLFQ significantly improve fairness and response time. MLFQ demonstrates the best balance for interactive workloads, albeit with reduced CPU utilization due to frequent preemption and I/O blocking. Although MLFQ shows higher turnaround time, it achieves superior response time and fairness, making it well-suited for mixed and interactive workloads.

13 Conclusion

In this project, a robust and extensible CPU scheduling simulator was successfully developed to model and analyze five widely studied scheduling algorithms: FCFS, SJF, Round Robin, Priority Scheduling with aging, and Multilevel Feedback Queue. By incorporating both basic and advanced scheduling features, the simulator provides a realistic approximation of operating system scheduling behavior while maintaining conceptual clarity.

Experimental evaluation across CPU-bound, I/O-bound, and mixed workloads revealed clear and meaningful trade-offs among the scheduling strategies. SJF consistently minimized turnaround and waiting times but suffered from reduced fairness. Round Robin and MLFQ significantly improved fairness and response time, making them more suitable for interactive workloads. Priority Scheduling, when combined with aging, effectively mitigated starvation while preserving predictable execution behavior. MLFQ demonstrated the best balance between responsiveness and fairness, albeit at the cost of increased turnaround time and reduced CPU utilization due to frequent preemption and I/O handling.

Overall, the results closely match theoretical expectations and validate the correctness of the simulator's design. This project not only reinforces core operating system concepts but also provides a practical framework for experimenting with scheduling policies and understanding their impact on system performance.

References

- [1] silberschatz, A., Galvin, P. B., & Gagne, G. "Operating System Concepts"
- [2] Tanenbaum, A. S. "Modern Operating Systems"
- [3] Stallings, W. "Operating Systems: Internals and Design Principles"